

Before we continue, note what we’ve done. First, we conjured up a simple, almost trivial example. We next explored this example a bit to uncover a design choice. Then, via some experimentation, we discovered an interesting consequence of one option and in the process learned something fundamental about the nature of our problem domain! The overall design process is a series of these little adventures. You don’t need any special license to do this sort of exploration, and you don’t need to be an expert in functional programming either. Just dive in and see what you find.

7.1.2 **Combining parallel computations**

Let’s see if we can avoid the aforementioned pitfall of combining `unit` and `get`. If we don’t call `get`, that implies that our `sum` function must return a `Par[Int]`. What consequences does this change reveal? Again, let’s just invent functions with the required signatures:

```
def sum(ints: IndexedSeq[Int]): Par[Int] =
  if (ints.size <= 1)
    Par.unit(ints.headOption getOrElse 0)
  else {
    val (l,r) = ints.splitAt(ints.length/2)
    Par.map2(sum(l), sum(r))(_ + _)
  }
```



EXERCISE 7.1

`Par.map2` is a new higher-order function for combining the result of two parallel computations. What is its signature? Give the most general signature possible (don’t assume it works only for `Int`).

Observe that we’re no longer calling `unit` in the recursive case, and it isn’t clear whether `unit` should accept its argument lazily anymore. In this example, accepting the argument lazily doesn’t seem to provide any benefit, but perhaps this isn’t always the case. Let’s come back to this question later.

What about `map2`—should it take its arguments lazily? It would make sense for `map2` to run both sides of the computation in parallel, giving each side equal opportunity to run (it would seem arbitrary for the order of the `map2` arguments to matter—we simply want `map2` to indicate that the two computations being combined are independent, and can be run in parallel). What choice lets us implement this meaning? As a simple test case, consider what happens if `map2` is strict in both arguments, and we’re evaluating `sum(IndexedSeq(1,2,3,4))`. Take a minute to work through and understand the following (somewhat stylized) program trace.

Listing 7.3 Program trace for sum

```

sum(IndexedSeq(1,2,3,4))
map2 (
  sum(IndexedSeq(1,2)),
  sum(IndexedSeq(3,4))) (_ + _)
map2 (
  map2 (
    sum(IndexedSeq(1)),
    sum(IndexedSeq(2))) (_ + _),
  sum(IndexedSeq(3,4))) (_ + _)
map2 (
  map2 (
    unit(1),
    unit(2)) (_ + _),
  sum(IndexedSeq(3,4))) (_ + _)
map2 (
  map2 (
    unit(1),
    unit(2)) (_ + _),
  map2 (
    sum(IndexedSeq(3)),
    sum(IndexedSeq(4))) (_ + _)) (_ + _)
...

```

In this trace, to evaluate `sum(x)`, we substitute `x` into the definition of `sum`, as we've done in previous chapters. Because `map2` is strict, and Scala evaluates arguments left to right, whenever we encounter `map2(sum(x), sum(y)) (_ + _)`, we have to then evaluate `sum(x)` and so on recursively. This has the rather unfortunate consequence that we'll strictly construct the entire left half of the tree of summations first before moving on to (strictly) constructing the right half. Here `sum(IndexedSeq(1,2))` gets fully expanded before we consider `sum(IndexedSeq(3,4))`. And if `map2` evaluates its arguments in parallel (using whatever resource is being used to implement the parallelism, like a thread pool), that implies the left half of our computation will start executing before we even begin constructing the right half of our computation.

What if we keep `map2` strict, but *don't* have it begin execution immediately? Does this help? If `map2` doesn't begin evaluation immediately, this implies a `Par` value is merely constructing a *description* of what needs to be computed in parallel. Nothing actually occurs until we *evaluate* this description, perhaps using a `get`-like function. The problem is that if we construct our descriptions strictly, they'll be rather heavy-weight objects. Looking back at our trace, our description will have to contain the full tree of operations to be performed:

```

map2 (
  map2 (
    unit(1),
    unit(2)) (_ + _),
  map2 (
    unit(3),
    unit(4)) (_ + _)) (_ + _)

```